Recursion and Fixed Points

Recursion is when a function calls itself.

Fixed-Point in Functional Analysis is a value that does not change under a given function.

Fixed Point Thm for Untyped Lambda Calculus

$$\forall L \in \Lambda, \exists M \in \Lambda, \text{s.t. } LM =_\beta M.x \notin FV(L)$$

$$\text{Witness: } (\lambda x.L(xx))(\lambda x.L(xx)) \to_\beta L((\lambda x.L(xx))(\lambda x.L(xx))) \implies LM =_\beta M$$

Recursion and Y-Combinator

Y-Combinator

$$Y = \lambda f.(\lambda x.f(xx))(\lambda x.f(xx))$$

$$\forall M, M \in \Lambda \implies YM =_\beta M(YM)$$

Defining a recursive function

If we have function $f = ...f...$, we can define $F = \lambda f....f....$ Then, we can define the recursive function as YF.

Factorial Function

Factorial is a function that takes a natural number n as input and returns the product of all positive integers less than or equal to n.
Intuitively, we can define the factorial function as follows:

$$fact = \lambda n.if\_then\_else\ (isZero\ n)\ 1\ (mul\ n\ (fact\ (pred\ n)))$$

Where *isZero* is a function that checks if a Church numeral is zero, *mul* is the multiplication function we defined earlier, and *pred* is the predecessor function that returns the Church numeral representing $n - 1$.

Factorial Function (Cont'd)

Step 1: Abstract out the *fact* function from the definition of *fact* to get F:

$$F = \lambda f.\lambda n.\mathit{if\_then\_else}\ (\mathit{isZero}\ n)\ 1\ (\mathit{mul}\ n\ (f\ (\mathit{pred}\ n)))$$

Notice that

$$F\ \mathit{fact} \rightarrow_\beta \mathit{fact}$$

Step 2: Solve the Fixed Point Equation, $F\ \mathit{fact} =_\beta \mathit{fact}$

$$YF = F(YF)$$

Let $M = YF$. Then, we have $FM =_\beta M$, which means M is a fixed point of F. So, we can know that

$$\mathit{fact} = YF$$